Data Types

While everything in Ruby is an object, some of the data types in Ruby have special syntax support, in particular for defining literal values. In the preceding examples, we used some simple strings and even string concatenation.

Strings

The previous example also showed some Ruby string objects. One way to create a string object is to use *string literals*, which are sequences of characters between single or double quotation marks. The difference between the two forms is the amount of processing Ruby does on the string while constructing the literal. In the single-quoted case, Ruby does very little. With only a few exceptions, what you type into the single-quoted string literal becomes the string's value.

With double-quotes, Ruby does more work. It looks for *substitutions*—sequences that start with a backslash character—and replaces them with a binary value. The most common of these is `\n`, which is replaced with a newline character. When you write a string containing a newline to the console, the `\n` forces a line break.

Then, Ruby performs *expression interpolation* in double-quoted strings. In the string, the sequence `#{expression}` is replaced by the value of `expression`. We could use this to rewrite our previous method:

```
def say_goodnight(name)
  "Good night, #{name.capitalize}"
end
puts say_goodnight('pa')
```

When Ruby constructs this string object, it looks at the current value of `name` and substitutes it into the string. Arbitrarily complex expressions are allowed in the `#{...}` construct. Here we invoked the `capitalize` method, defined for all strings, to output our parameter with a leading uppercase letter.